

Digital Electronics

COMPLETE UNIT NOTES

UNIT 2

Number Systems

Complements

Boolean Algebra

K-Maps

Combinational Circuits

Universal Gates

Definitions · worked examples · diagrams · practice questions with answer key

Contents

1. Number Systems and Positional Notation	3
2. Radix (Base)	3
3. Radix Interconversion	4
4. Binary Arithmetic	7
5. Radix and Diminished-Radix Complements	7
6. Boolean Algebra Fundamentals	9
7. Basic Boolean Algebra Laws and Theorems	9
8. Boolean Functions and Truth Tables	11
9. Boolean Function Minimization	11
10. Karnaugh Maps (K-Maps)	11
11. Combinational Circuits and Their Analysis	14
12. Logic Gates	15
13. Universal Gates	16
14. Realization of Primary Gates Using NAND Only	16
15. Realization of Primary Gates Using NOR Only	17
16. Quick Revision Tables	17
17. Common Mistakes to Avoid	19
18. Practice Questions	20
19. Answer Key	22

How to use these notes. Read each section in order, study the diagrams, then close the notes and work through the practice questions in Section 18. Check yourself against the answer key in Section 19.

1

Number Systems and Positional Notation

A **number system** is a way of representing quantities using a set of symbols and rules. Digital electronics commonly uses four of them. Digital circuits work with two logical states, which is why **binary** is fundamental.

Decimal 10 BASE / RADIX Digits 0 1 2 3 4 5 6 7 8 9 Everyday counting	Binary 2 BASE / RADIX Digits 0 and 1 Two logic states	Octal 8 BASE / RADIX Digits 0 through 7 Compact binary (3 bits)	Hexadecimal 16 BASE / RADIX Digits 0-9 and A-F Compact binary (4 bits)
--	---	---	--

Hex letters: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15

Positional value

Each digit has a value decided by its **position**. In a base-*r* number, positions to the left of the radix point carry the powers $r^0, r^1, r^2, \dots$ and positions to the right carry $r^{-1}, r^{-2}, r^{-3}, \dots$. The **radix point** separates the integer part from the fractional part.

	integer part				fractional part		
Weight	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	
Digit	1	0	1	1	.	01	
Place value	8	4	2	1	radix point	0.5	0.25
Digit × value	8	0	2	1		0	0.25
$8 + 0 + 2 + 1 + 0 + 0.25 = (11.25)_{10}$							

Fig. 1 — Positional value of $(1011.01)_2 = (11.25)_{10}$

2

Radix (Base)

The **radix** is the number of distinct digits available in a positional number system, **including zero**. For a base-*r* number, each position represents a power of *r*.

System	Radix	Digits available
Decimal	10	0 - 9
Binary	2	0, 1
Octal	8	0 - 7
Hexadecimal	16	0 - 9, A - F

REMEMBER

A valid digit must always be **smaller than the base**:

- Binary permits only 0 and 1.
- Octal does not permit 8 or 9.
- Decimal does not permit A, B, C ... as ordinary digits.

3 Radix Interconversion

Binary is the hub: octal and hexadecimal are just compact ways of writing binary, and decimal is reached by place values or repeated division.

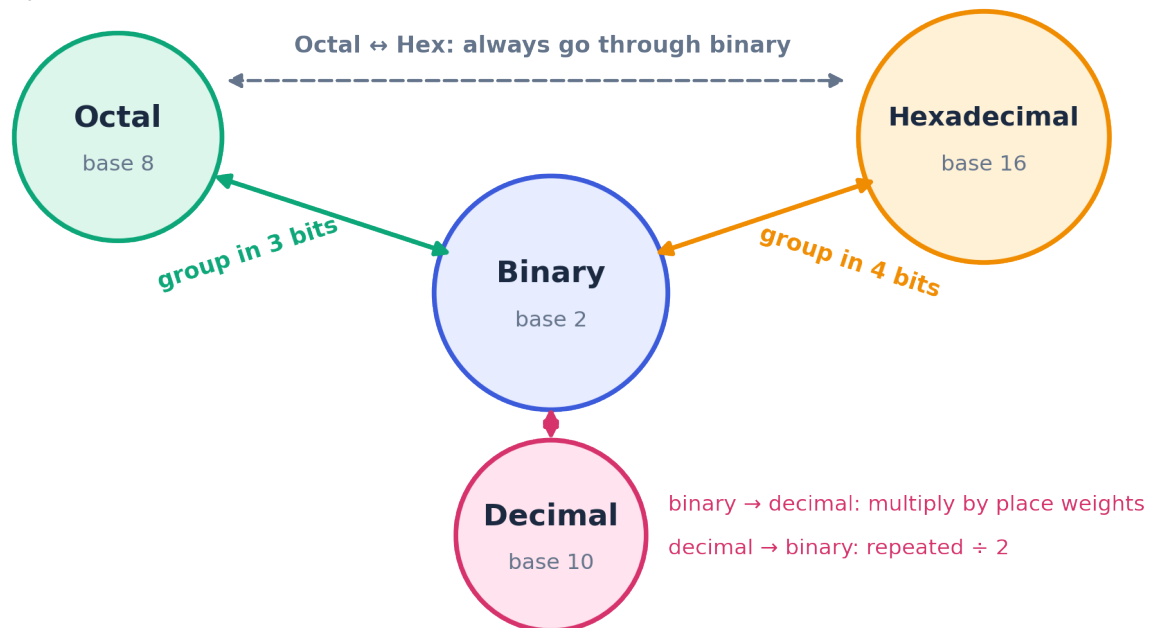


Fig. 2 — Conversion map — binary sits in the middle

3.1 Any base → decimal

Multiply each digit by its positional weight and add the results.

EXAMPLE: $(1101.101)_2$ TO DECIMAL

$$\begin{aligned}
 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\
 &= 8 + 4 + 0 + 1 + 0.5 + 0 + 0.125 = \mathbf{(13.625)_{10}}
 \end{aligned}$$

EXAMPLE: $(572.4)_8$ TO DECIMAL

$$\begin{aligned}
 &= 5 \times 8^2 + 7 \times 8^1 + 2 \times 8^0 + 4 \times 8^{-1} \\
 &= 320 + 56 + 2 + 0.5 = \mathbf{(378.5)_{10}}
 \end{aligned}$$

3.2 Decimal integer → another base

Repeatedly **divide** the integer by the target base and record each remainder. Read the remainders from **bottom to top**.

3.3 Decimal fraction → another base

Repeatedly **multiply** the fractional part by the target base. Record the integer part of each product in the order obtained. Continue until the fraction becomes zero or the required precision is reached.

Integer: repeated division $(83)_{10} \rightarrow \text{octal}$

$83 \div 8 = 10 \text{ remainder } 3$

$10 \div 8 = 1 \text{ remainder } 2$

$1 \div 8 = 0 \text{ remainder } 1$

$(83)_{10} = (123)_8$

3**2****1**

read upward

Fraction: repeated multiplication $(0.625)_{10} \rightarrow \text{binary (keep the integer part)}$

$0.625 \times 2 = 1.25$

$0.25 \times 2 = 0.5$

$0.5 \times 2 = 1.0$

$(0.625)_{10} = (0.101)_2$

1**0****1**

read downward

Fig. 3 — Division for integers (read up), multiplication for fractions (read down)**COMBINE BOTH PARTS** $(10.625)_{10}$ to binary: 10 in binary is 1010 and 0.625 is .101, so the answer is **$(1010.101)_2$** .**3.4 Binary → octal**

Group the bits in sets of **three**. Integer part: group from the radix point towards the **left**. Fractional part: group from the radix point towards the **right**. Add zeros at the outside ends if needed, then convert each group to one octal digit.

Bits	000	001	010	011	100	101	110	111
Octal	0	1	2	3	4	5	6	7

3.5 Octal → binary

Replace each octal digit with its 3-bit binary equivalent. **$(572.4)_8 = 101\ 111\ 010 . 100 = (101111010.100)_2$**

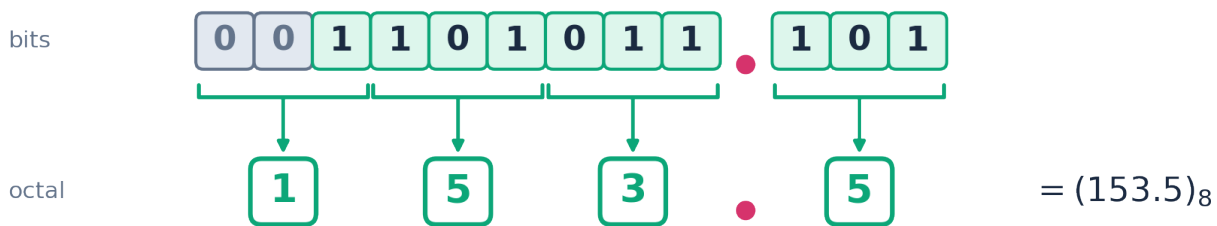
3.6 Binary → hexadecimal

Group the bits in sets of **four**, using the same grouping direction as for octal, and pad with zeros at the outside ends if necessary.

Bits	0000	0001	0010	0011	0100	0101	0110	0111
Hex	0	1	2	3	4	5	6	7
Bits	1000	1001	1010	1011	1100	1101	1110	1111
Hex	8	9	A	B	C	D	E	F

Binary → Octal (groups of 3)

$(1101011.101)_2 \rightarrow \text{pad } 001\ 101\ 011\ .\ 101$



grey boxes = padding zeros (value unchanged)

Binary → Hexadecimal (groups of 4)

$(11010110.1011)_2 \rightarrow 1101\ 0110\ .\ 1011$



Fig. 4 — Grouping bits — threes for octal, fours for hexadecimal

3.7 Hexadecimal → binary

Replace every hex digit with its 4-bit binary equivalent. $(B4.C)_{16} = 1011\ 0100\ .\ 1100 = (10110100.1100)_2$

3.8 Octal ↔ hexadecimal

Convert through binary. **Octal** → binary (3-bit groups) → regroup into 4-bit groups → **hex**. **Hex** → binary (4-bit groups) → regroup into 3-bit groups → **octal**.

EXAMPLE: $(3A)_{16}$ TO OCTAL

$3A = 0011\ 1010_2$

Group in threes: 000 111 010

= $(72)_8$

3.9 Hexadecimal → decimal

EXAMPLE: $(B4.C)_{16}$

$= 11 \times 16^1 + 4 \times 16^0 + 12 \times 16^{-1}$

$= 176 + 4 + 0.75 = (180.75)_{10}$

IMPORTANT CONVERSION RULES

- Do not use digits that are invalid for the base.
- Binary → octal: group in **3s**. Binary → hex: group in **4s**.
- Fractional bits are grouped moving **away from the radix point to the right**.
- Padding zeros do not change the value.
- Keep the radix point in the correct place.

4 Binary Arithmetic

4.1 Binary addition

Sum	Result	Meaning
0 + 0	0	
0 + 1	1	
1 + 0	1	
1 + 1	10	write 0, carry 1
1 + 1 + 1	11	write 1, carry 1

$$\begin{array}{r}
 \textcolor{red}{1} \quad \textcolor{red}{1} \quad \textcolor{red}{1} \quad \textcolor{red}{0} \\
 \quad 1 \quad 0 \quad 1 \quad 1 \\
 + \quad 0 \quad 1 \quad 1 \quad 0 \\
 \hline
 \textcolor{blue}{1} \quad \textcolor{blue}{0} \quad \textcolor{blue}{0} \quad \textcolor{blue}{0} \quad \textcolor{blue}{1} \\
 \hline
 \mathbf{1011 + 0110 = 10001}
 \end{array}$$

Work right to left and include any carry.

4.2 Binary subtraction

Difference	Result
0 - 0	0
1 - 0	1
1 - 1	0
0 - 1	borrow needed

$$\begin{array}{r}
 \textcolor{red}{0} \quad \textcolor{red}{1} \quad \textcolor{red}{1} \quad \textcolor{red}{10} \\
 \quad 1 \quad 0 \quad 0 \quad 0 \\
 - \quad 0 \quad 0 \quad 1 \quad 1 \\
 \hline
 \textcolor{blue}{0} \quad \textcolor{blue}{1} \quad \textcolor{blue}{0} \quad \textcolor{blue}{1} \\
 \hline
 \mathbf{1000 - 0011 = 0101}
 \end{array}$$

Red row = digits after borrowing; the borrow passes across the zero columns.

KEY IDEA

In binary, borrowing 1 from the next column gives 10_2 in the current column, so $10_2 - 1_2 = 1_2$. Borrowing can pass across one or more zero columns.

5 Radix and Diminished-Radix Complements

For a base r , the **diminished-radix complement** is the $(r - 1)$'s complement and the **radix complement** is the r 's complement. For a fixed n -digit number N :

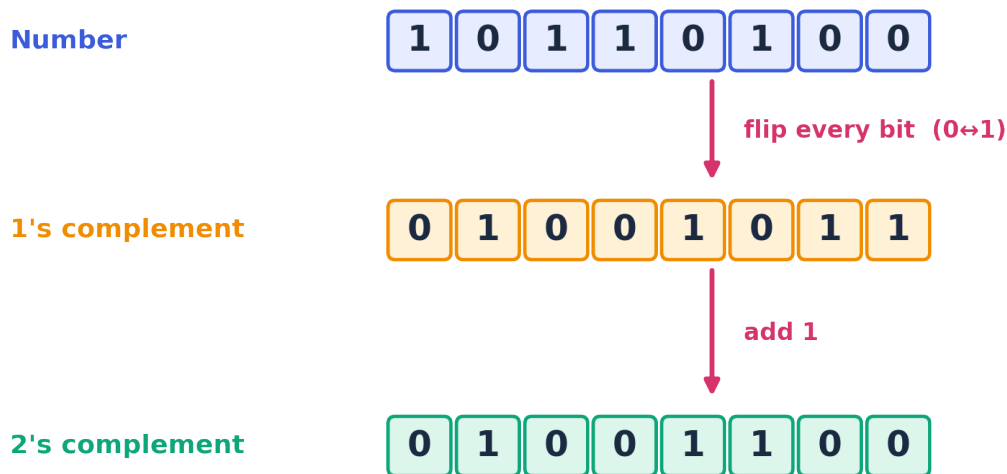
$$(r - 1)\text{'s complement} = (r^n - 1) - N$$

$$r\text{'s complement} = r^n - N = (r - 1)\text{'s complement} + 1$$

Always preserve the specified number of digits / bits.

5.1 Binary complements (radix 2)

The diminished-radix complement is the **1's complement** (flip every bit). The radix complement is the **2's complement** (1's complement + 1).



2's complement = 1's complement + 1 (keep the same number of bits)

Fig. 5 — 1's and 2's complement of 10110100

ANOTHER EXAMPLE: 11001010

Flip bits: 00110101

Add 1: 00110110 → 2's complement = **00110110**

5.2 – 5.4 Decimal, octal and hexadecimal

Decimal — 9's and 10's				
Number	4	6	8	2
9 – digit	5	3	1	7
10's (+1)	5	3	1	8

Octal — 7's and 8's				
Number	3	4	7	
7 – digit	4	3	0	
8's (+1)	4	3	1	

Hexadecimal — F's and 16's				
Number	2	B	4	
F – digit	D	4	B	
16's (+1)	D	4	C	

How to read these				
Step 1 — subtract every digit from (r – 1): 9, 7 or F (=15).				
Step 2 — add 1 to get the radix complement.				
Hex check: F–2=D, F–B=4, F–4=B.				

5.5 Why complements are useful

Complements are used to **represent negative numbers** and to **perform subtraction using addition-based digital circuits**. For this syllabus, know how to calculate complements in binary, decimal, octal and hexadecimal.

COMMON COMPLEMENT MISTAKE

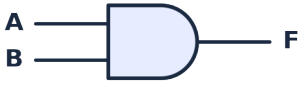
Do not confuse the diminished-radix complement with the radix complement. The radix complement is the diminished-radix complement **plus 1**.

6 Boolean Algebra Fundamentals

Boolean algebra is a mathematical system used to describe logic circuits. A Boolean variable can have only two values: **0** = FALSE / OFF / LOW and **1** = TRUE / ON / HIGH.

Three basic operations: **AND**, written $A \cdot B$ or AB ; **OR**, written $A + B$; **NOT**, written $\bar{A}$.

AND



$F = A \cdot B$

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

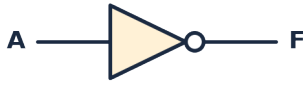
OR



$F = A + B$

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

NOT



$F = \bar{A}$

A	$\bar{A}$
0	1
1	0

AND outputs 1 only when every input is 1. **OR** outputs 1 when at least one input is 1. **NOT** flips the input.

IMPORTANT

Boolean **+** means **OR**, not ordinary arithmetic addition. Boolean multiplication means **AND**.

7 Basic Boolean Algebra Laws and Theorems

Let A and B be Boolean variables.

Law	Form 1	Form 2
Identity	$A + 0 = A$	$A \cdot 1 = A$
Domination (null)	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Complement	$A + \bar{A} = 1$	$A \cdot \bar{A} = 0$
Involution	$\bar{\bar{A}} = A$	—
Commutative	$A + B = B + A$	$A \cdot B = B \cdot A$
Associative	$(A + B) + C = A + (B + C)$	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$
Distributive	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan's	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \bar{B}$

The first distributive law is like ordinary algebra; the second is also a Boolean identity and is very useful in simplification. A related absorption identity: $A + \bar{A}B = A + B$.

7.10 De Morgan's laws

A bar over an expression can be moved inside by changing AND to OR (or OR to AND) and complementing every variable. In gate form, **NAND** is an OR gate with inverted inputs, and **NOR** is an AND gate with inverted inputs.

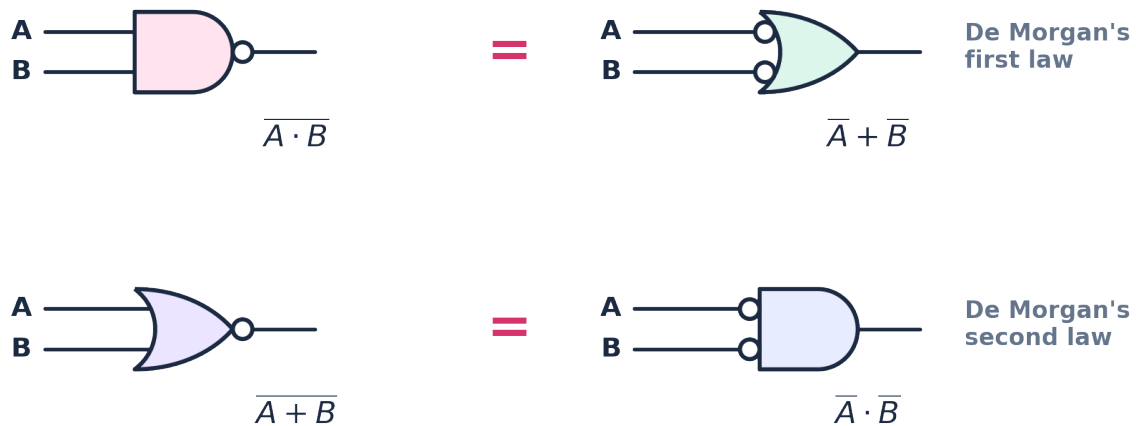


Fig. 6 — De Morgan's laws as gate equivalents

DE MORGAN'S MEMORY TRICK

$$\overline{AB} = \overline{A} + \overline{B} \quad \overline{A + B} = \overline{A} \overline{B} \quad \overline{ABC} = \overline{A} + \overline{B} + \overline{C}$$

Memory trick: **break the bar, change the sign.**

7.11 Simplification examples

Example 1

$$\begin{aligned} F &= A + AB \\ &= A(1 + B) && \text{factor out A} \\ &= A \cdot 1 && 1 + B = 1 \\ &= A && \text{identity} \end{aligned}$$

Example 2

$$\begin{aligned} F &= AB + \overline{A}B \\ &= B(A + \overline{A}) && \text{factor out B} \\ &= B \cdot 1 && \text{complement} \\ &= B && \text{identity} \end{aligned}$$

Example 3

$$\begin{aligned} F &= A + \overline{A}B \\ &= (A + \overline{A})(A + B) && \text{2nd distributive} \\ &= 1 \cdot (A + B) && \text{complement} \\ &= A + B && \text{identity} \end{aligned}$$

Example 4

$$\begin{aligned} F &= AB + \overline{A}B \\ &= A(B + \overline{B}) && \text{factor out A} \\ &= A \cdot 1 && \text{complement} \\ &= A && \text{identity} \end{aligned}$$

8

Boolean Functions and Truth Tables

A **Boolean function** describes the output of a logic circuit in terms of its Boolean inputs, for example $F = A \cdot B + C$. A **truth table** lists every possible input combination and the corresponding output.

KEY IDEA

With n binary inputs there are 2^n input combinations: 2 inputs $\rightarrow$ 4 rows, 3 inputs $\rightarrow$ 8 rows.

Example: $F = A \cdot B + C$

A	B	C	AB	F
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Method to analyse an expression

- List every input combination.
- Evaluate any NOT operations.
- Evaluate AND operations.
- Evaluate OR operations.
- Record the final output.

Follow brackets and operation order carefully.

9

Boolean Function Minimization

Minimization means simplifying a Boolean expression while keeping the same output for every possible input combination.

Fewer gates

Fewer gates may be needed.

Simpler design

Circuit design becomes simpler.

Lower cost

Hardware cost and complexity may be reduced.

Common methods in this unit: (1) Boolean algebra laws (2) Karnaugh maps (K-maps)

EXAMPLE

$$F = AB + \bar{A}B = B(A + \bar{A}) = B \cdot 1 = B$$

The original expression uses multiple terms, but the simplified result is just **B**.

$F = A + AB = A$ by the absorption law.

10

Karnaugh Maps (K-Maps)

A **Karnaugh map** is a visual method for minimizing Boolean functions using the truth-table output values.

10.1 Main rules

- 1 Put a **1** in every cell where the function output is 1.
- 2 Group adjacent 1s in groups of **1, 2, 4, 8, ...** cells.
- 3 Make groups as **large** as possible when it helps cover the 1s.
- 4 Every required 1 must be covered by **at least one** group.
- 5 Groups **may overlap** if that gives a simpler expression.
- 6 The map **wraps around**: a cell can group with the cell across the opposite edge.
- 7 **Diagonal** cells are **not** adjacent.
- 8 In each group **keep the variables that stay constant** and remove those that change.

READING A GROUP

Constant variable = **1** → write it **uncomplemented** (A). Constant variable = **0** → write it **complemented** ($\bar{A}$).

10.2 Two-variable K-map

Rows represent A and columns represent B. The cell labels are the minterms:

Cell	m0	m1	m2	m3
Term	$\bar{A}\bar{B}$	$\bar{A}B$	$A\bar{B}$	AB
A, B	0, 0	0, 1	1, 0	1, 1

10.3 Two-variable examples

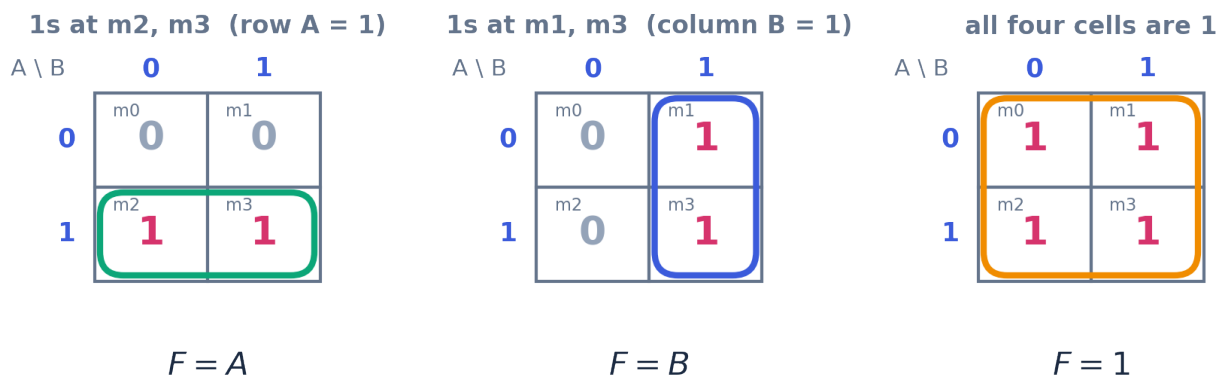


Fig. 7 — Two-variable K-maps: the variable that changes inside the group disappears

Example 1: 1s at m2, m3 → they fill the A = 1 row; A stays 1 while B changes, so **F = A**. Example 2: 1s at m1, m3 → the B = 1 column; B stays 1 while A changes, so **F = B**. Example 3: all four cells are 1 → **F = 1**.

10.4 Three-variable K-map

A common arrangement has **A** as the row variable and **BC** as the column pair. The columns use **Gray-code order**: **00, 01, 11, 10**, so that neighbouring columns differ in only one bit.

column order is Gray code

A \ BC	00	01	11	10
0	m0 000	m1 001	m3 011	m2 010
1	m4 100	m5 101	m7 111	m6 110

neighbouring columns differ in only one bit,
and the map wraps around its edges

Fig. 8 — Three-variable K-map layout with minterm numbers

For a group, find the variables that do not change and eliminate the ones that do. Three illustrations:

(a) wrap-around

A\BC	00	01	11	10
0	1	0	0	1
1	0	0	0	0

$m0 + m2 \rightarrow F = \overline{A}\overline{C}$
B changes, A and C stay 0

(b) group of four

A\BC	00	01	11	10
0	0	0	0	0
1	1	1	1	1

$m4 + m5 + m6 + m7 \rightarrow F = A$
B and C change, A stays 1

(c) diagonal $\neq$ adjacent

A\BC	00	01	11	10
0	1	0	0	0
1	0	1	0	0

$m0$ and $m5$ each need their own group
cannot be one group

Fig. 9 — Grouping examples on a three-variable map (illustrative functions)

10.5 K-map exam checklist

1	Fill the map correctly from the truth table or minterm list.
2	Use Gray-code order , not ordinary 00, 01, 10, 11 order.
3	Group 1s in powers of two .
4	Prefer larger valid groups.
5	Include all 1s.
6	Write one simplified term per group .
7	OR the terms together.

11

Combinational Circuits and Their Analysis

A **combinational circuit** is a logic circuit whose output depends **only on the current input values**. It does not depend on a stored previous state. Examples include adders, subtractors, multiplexers, demultiplexers, encoders and decoders.

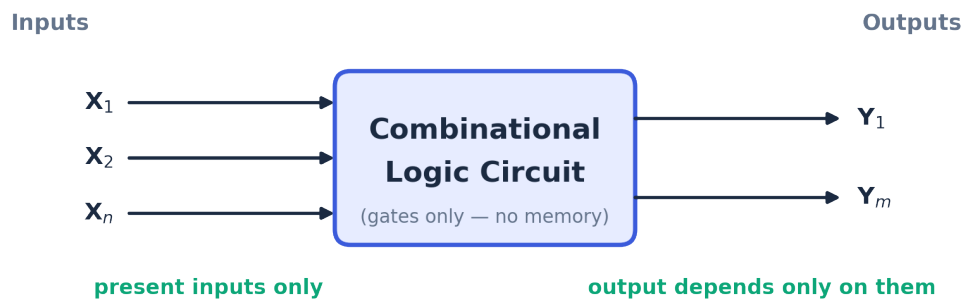


Fig. 10 — Block view of a combinational circuit

11.1 Analysis procedure

- 1 Identify the inputs and the output.
- 2 Read the circuit from input to output.
- 3 Write an expression for each intermediate gate.
- 4 Combine the intermediate expressions.
- 5 Make a truth table for every input combination.
- 6 Evaluate the final output for each row.

11.2 – 11.3 Gate followed by NOT

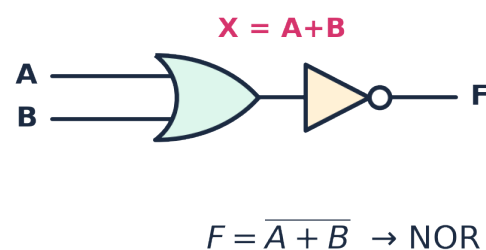
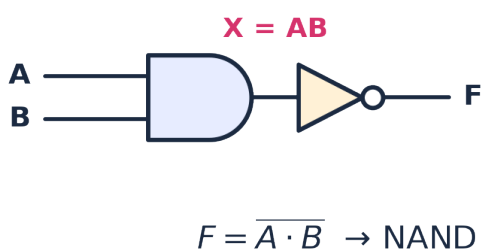
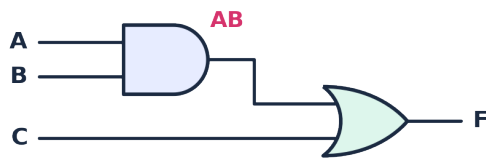


Fig. 11 — AND then NOT gives NAND; OR then NOT gives NOR

A	B	AB	F
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

A	B	A+B	F
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

11.4 Expression example: $F = AB + C$



$$F = AB + C$$

- Calculate **AB** first.
- OR that result with **C**.
- If $AB = 0$ and $C = 1$, then $F = 1$.
- If $AB = 1$ and $C = 0$, then $F = 1$.
- The full truth table is in Section 8.

12

Logic Gates

A **logic gate** performs a Boolean operation. The primary gates are AND ($F = AB$), OR ($F = A + B$) and NOT ($F = \bar{A}$). Two further gates are built by negating the output of AND and OR.

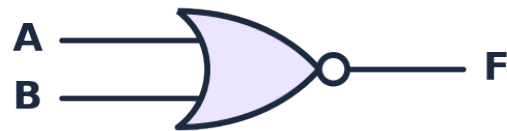
NAND (NOT-AND)



$$F = \overline{AB}$$

A	B	NAND
0	0	1
0	1	1
1	0	1
1	1	0

NOR (NOT-OR)



$$F = \overline{A + B}$$

A	B	NOR
0	0	1
0	1	0
1	0	0
1	1	0

NAND VS NOR IN ONE LINE

NAND output is **0** only when both inputs are **1**.

NOR output is **1** only when both inputs are **0**.

12.4 Side-by-side comparison

Input A B	NAND	NOR
0 0	1	1
0 1	1	0
1 0	1	0
1 1	0	0

13 Universal Gates

NAND and **NOR** are called **universal gates** because either gate type *alone* can be used to construct NOT, AND and OR. Since these operations can express any Boolean function, either NAND alone or NOR alone can implement **any** Boolean logic function.

NAND = NOT-AND

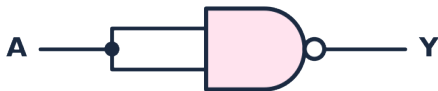
$\overline{AB}$
builds NOT, AND, OR

NOR = NOT-OR

$\overline{A + B}$
builds NOT, AND, OR

14 Realization of Primary Gates Using NAND Only

14.1 NOT using NAND



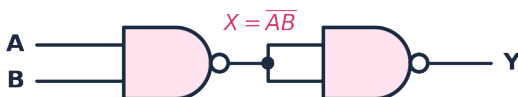
$$Y = \overline{A \cdot A} = \overline{A}$$

1 NAND gate required

Connect **both inputs** of one NAND gate to A.

$$Y = \overline{A \cdot A} = \overline{A}$$

14.2 AND using NAND



$$Y = \overline{X \cdot X} = \overline{X} = AB$$

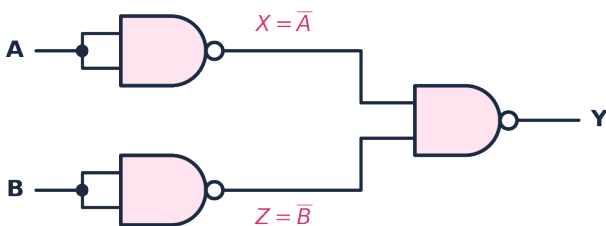
2 NAND gates required

First NAND: $X = \overline{AB}$

The second NAND has both inputs connected to X:

$$Y = \overline{X \cdot X} = \overline{X} = AB$$

14.3 OR using NAND



$$Y = \overline{\overline{A} \cdot \overline{B}} = A + B$$

3 NAND gates required

Two NAND gates act as inverters: $X = \overline{A \cdot A} = \overline{A}$ and $Z = \overline{B \cdot B} = \overline{B}$.

Feed $\overline{A}$ and $\overline{B}$ into a third NAND:

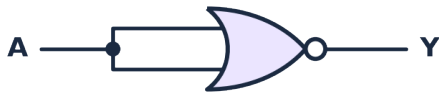
$$Y = \overline{\overline{A} \cdot \overline{B}} = A + B$$

(by De Morgan's law)

15

Realization of Primary Gates Using NOR Only

15.1 NOT using NOR

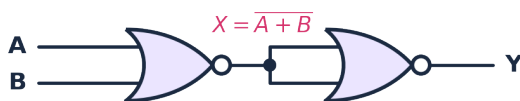


$$Y = \overline{A + A} = \overline{A}$$

1 NOR gate required

Connect **both inputs** of one NOR gate to A.
 $Y = \overline{A + A} = \overline{A}$

15.2 OR using NOR



$$Y = \overline{\overline{X} + \overline{X}} = \overline{\overline{X}} = X = A + B$$

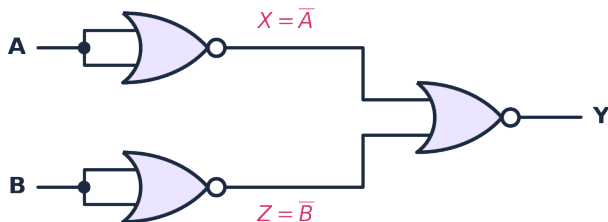
2 NOR gates required

First NOR: $X = \overline{A + B}$
 The second NOR has both inputs connected to X:
 $Y = \overline{\overline{X} + \overline{X}} = \overline{\overline{X}} = X = A + B$

GATE-COUNT CORRECTION

OR using NOR needs TWO NOR gates, not three. Three gates are needed for **AND** using NOR.

15.3 AND using NOR



$$Y = \overline{\overline{A} + \overline{B}} = AB$$

3 NOR gates required

Two NOR gates act as inverters: $X = \overline{A + A} = \overline{A}$ and $Z = \overline{B + B} = \overline{B}$.
 Feed X and Z into a third NOR:
 $Y = \overline{\overline{A} + \overline{B}} = AB$
 (by De Morgan's law)

16

Quick Revision Tables

16.1 Number systems

System	Base	Valid digits
Binary	2	0, 1
Octal	8	0 - 7
Decimal	10	0 - 9
Hexadecimal	16	0 - 9, A - F

Conversion	Group size
Binary → octal	3 bits
Octal → binary	3 bits per octal digit
Binary → hexadecimal	4 bits
Hexadecimal → binary	4 bits per hex digit

16.3 Complement names

Base	Diminished-radix complement	Radix complement
2	1's complement	2's complement
8	7's complement	8's complement
10	9's complement	10's complement
16	F's complement	16's complement

Radix complement = diminished-radix complement + 1.

16.4 Boolean laws

Law	Expressions
Identity	$A + 0 = A$; $A \cdot 1 = A$
Domination	$A + 1 = 1$; $A \cdot 0 = 0$
Idempotent	$A + A = A$; $A \cdot A = A$
Complement	$A + \bar{A} = 1$; $A \cdot \bar{A} = 0$
Involution	$\overline{\bar{A}} = A$
Absorption	$A + AB = A$; $A(A + B) = A$
De Morgan	$\overline{AB} = \bar{A} + \bar{B}$; $\overline{\bar{A} + \bar{B}} = \bar{A} \bar{B}$

16.5 Gate truth tables

A	B	AND	OR	NAND	NOR
0	0	0	0	1	1
0	1	0	1	1	0
1	0	0	1	1	0
1	1	1	1	0	0

16.6 Primary gates using universal gates

Gate to create	NAND-only gates	NOR-only gates
NOT	1	1
AND	2	3
OR	3	2

17

Common Mistakes to Avoid

- 1 Treating Boolean **+** as ordinary arithmetic addition. In Boolean algebra, **+** means **OR**.
- 2 Forgetting that Boolean multiplication means **AND**.
- 3 Confusing NAND with NOR: **NAND = NOT(AND)**; **NOR = NOT(OR)**.
- 4 Forgetting to **add 1** when finding a radix complement.
- 5 Reading division remainders from top to bottom instead of **bottom to top** when converting a decimal integer to another base.
- 6 Grouping binary-to-octal digits in **fours** instead of threes.
- 7 Grouping binary-to-hex digits in **threes** instead of fours.
- 8 Grouping fractional bits in the wrong direction.
- 9 Using ordinary binary order instead of Gray-code order in a 3-variable K-map. Correct column order: **00, 01, 11, 10**.
- 10 Grouping **diagonal** K-map cells. Diagonal cells are not adjacent.
- 11 Making K-map groups that are **not powers of two**.
- 12 Forgetting that K-map **edges wrap around**.
- 13 In a K-map group, keeping variables that change and removing variables that stay constant. Do the opposite: **keep the constant ones**.
- 14 Assuming a combinational circuit depends on previous inputs. It depends **only on current inputs**.
- 15 Incorrect universal-gate counts. NAND-only: NOT 1, AND 2, OR 3. NOR-only: NOT 1, AND 3, OR 2.

18

Practice Questions

Try these **without looking at the answer key**.

A. Number systems and conversions

1	Convert $(101101)_2$ to decimal.
2	Convert $(45)_{10}$ to binary.
3	Convert $(736)_8$ to binary.
4	Convert $(11011101)_2$ to hexadecimal.
5	Convert $(3A)_{16}$ to octal.
6	Convert $(572.4)_8$ to decimal.
7	Convert $(10.625)_{10}$ to binary.
8	Convert $(B4.C)_{16}$ to decimal.

B. Binary arithmetic

9	Add: $1011 + 0110$.
10	Add: $1101 + 1011$.
11	Subtract: $101 - 010$.
12	Subtract: $1000 - 0011$.

C. Complements

13	Find the 1's complement of 11010110 .
14	Find the 2's complement of 10110100 .
15	Find the 9's complement of 4682 .
16	Find the 10's complement of 4682 .
17	Find the 7's complement of $(347)_8$.
18	Find the 8's complement of $(347)_8$.
19	Find the F's complement of $(2B4)_{16}$.
20	Find the 16's complement of $(2B4)_{16}$.

D. Boolean algebra

21	Simplify $A + AB$.
22	Simplify $AB + \overline{A}B$.
23	State the identity laws.
24	State both De Morgan's laws.
25	Simplify $A + \overline{A}B$.
26	Simplify $AB + A\overline{B}$.

E. Truth tables and gates

27	For $F = AB$, find F when $A = 1$, $B = 0$.
28	For $F = A+B$, find F when $A = 0$, $B = 1$.
29	Find the NAND output for $A = 1$, $B = 1$.
30	Find the NOR output for $A = 0$, $B = 0$.
31	How many input combinations are there for a 3-input truth table?
32	Write the Boolean expression for an AND gate followed by NOT.
33	Write the Boolean expression for an OR gate followed by NOT.

F. K-maps and minimization

34	What group sizes are allowed in a K-map?
35	Are diagonal cells adjacent?
36	What is the Gray-code order for 3-variable K-map columns?
37	In a two-variable K-map, if the two 1s occupy the row $A = 1$, what is the simplified expression?
38	In a two-variable K-map, if the two 1s occupy the column $B = 1$, what is the simplified expression?

G. Universal gates

39	Name the two universal gates.
40	How many NAND gates are needed to make NOT?
41	How many NAND gates are needed to make AND?
42	How many NAND gates are needed to make OR?
43	How many NOR gates are needed to make NOT?
44	How many NOR gates are needed to make AND?
45	How many NOR gates are needed to make OR?
46	Explain why NAND and NOR are called universal gates.

19

Answer Key

1	45	24	$\overline{AB} = \overline{A} + \overline{B}$; $\overline{A + B} = \overline{A} \cdot \overline{B}$
2	101101	25	$A + B$
3	111011110	26	A
4	DD	27	0
5	72	28	1
6	378.5	29	0
7	1010.101	30	1
8	180.75	31	8
9	10001	32	$F = \overline{AB}$ (NAND)
10	11000	33	$F = \overline{A + B}$ (NOR)
11	011	34	1, 2, 4, 8, ... cells
12	0101	35	No
13	00101001	36	00, 01, 11, 10
14	01001100	37	$F = A$
15	5317	38	$F = B$
16	5318	39	NAND and NOR
17	$(430)_8$	40	1
18	$(431)_8$	41	2
19	$(D4B)_{16}$	42	3
20	$(D4C)_{16}$	43	1
21	A	44	3
22	B	45	2
23	$A + 0 = A$ and $A \cdot 1 = A$	46	Each can be used alone to construct NOT, AND and OR, and therefore any Boolean logic function.

EXAM REVISION ADVICE

Read the definitions and formulas first. Then solve the practice questions **without checking the answers**. Spend extra practice time on number-base conversions, complements, Boolean simplification, K-map grouping, truth tables and universal-gate realization.

Being familiar with the notes is useful, but being able to **solve questions independently** is the real test of readiness.